

Autonomous Drone System

TEAM:

Arwa Hamdy Mohammdy

Basmala Ayman omar

Samir Fawzy Ahmed

Omar Diaaeldeen Fawzy

Nour Tamer Mustafa

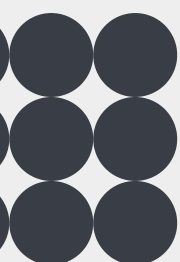


Objective

To develop an intelligent autonomous drone navigation system that combines Computer Vision and Reinforcement Learning to navigate safely and efficiently in dynamic environments without human intervention.

Key Objectives

- ✓ Object Detection using YOLO
- ✓ Real-Time Environment Understanding
- ✓ Intelligent Decision Making (Dueling DQN& PPO)
- ✓ Safe Obstacle Avoidance
- ✓ Autonomous Mission Execution



Problem Statement

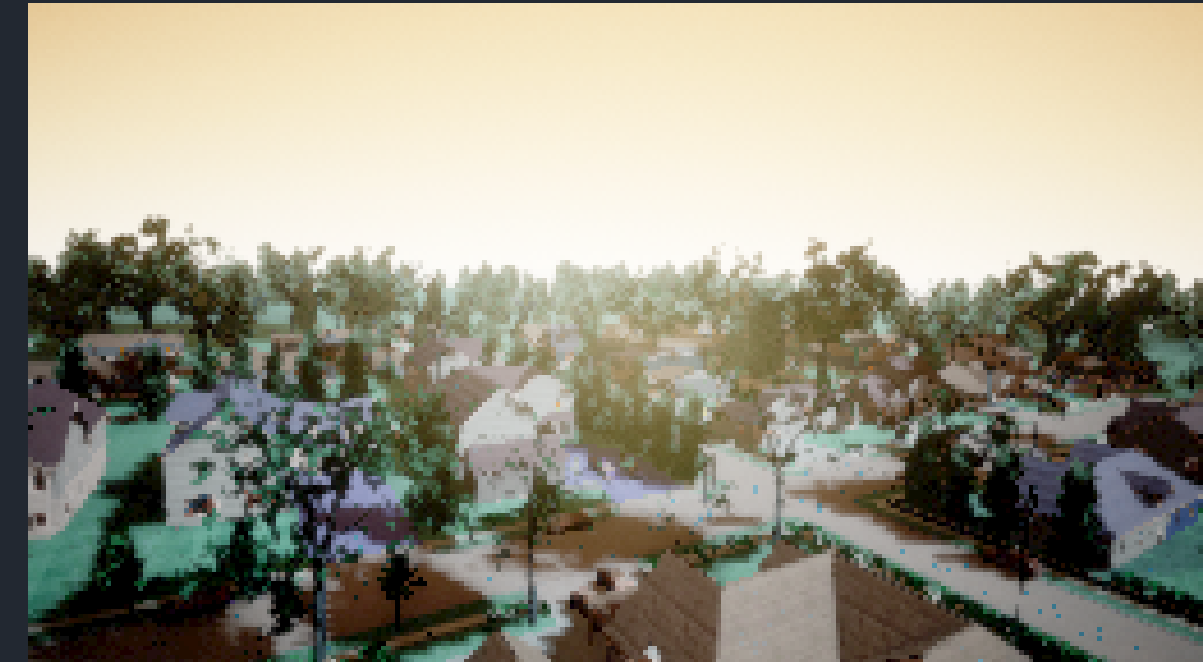
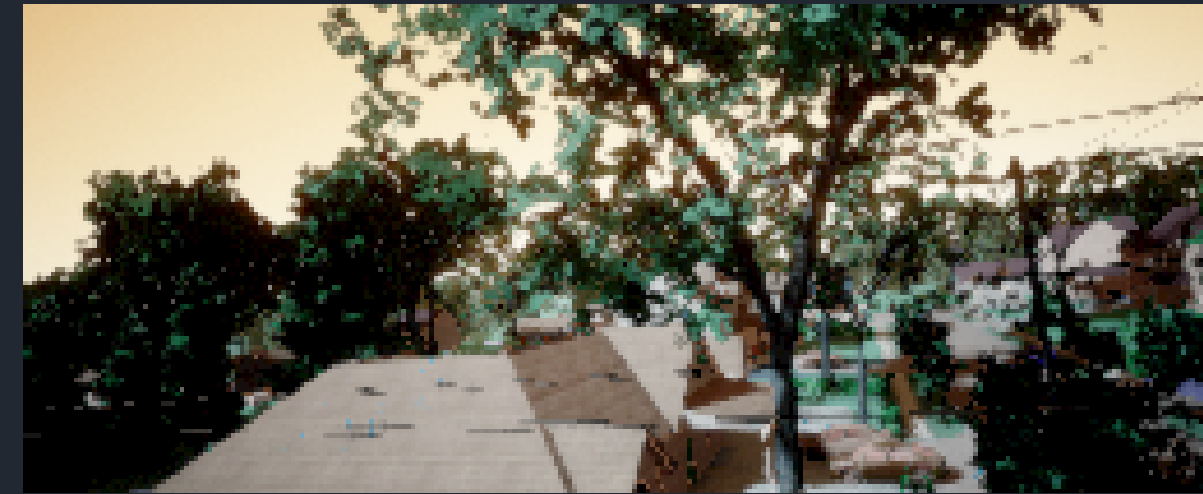
Traditional drone navigation systems face several challenges in real-world environments:

01. Limited Environmental Awareness

02. Static Navigation

03. Real-Time Decision Making

04. Safe Obstacle Avoidance



PROBLEM



- Manual drone control requires continuous human attention.
- High risk of collisions in dynamic environments.
- GPS-only navigation cannot react to unexpected obstacles.
- Traditional obstacle avoidance is limited and inflexible.
- Need for safe, autonomous drone navigation.

SOLUTION



- ✓ Autonomous drone navigation system.
- ✓ YOLO-based real-time object detection.
- ✓ Reinforcement Learning for decision making.
- ✓ Path Planner for safe obstacle avoidance.
- ✓ Flight Narrator to explain every navigation decision.

KEY TECHNOLOGIES

Python
OpenCV
YOLOv8
Reinforcement Learning
AirSim
PyTorch
Async Programming



UNIQUE VALUE



Why Our System Is Different

- Combines Computer Vision with Reinforcement Learning.
- Makes intelligent decisions in real time.
- Avoids obstacles before collisions occur.
- Explains every drone action using Flight Narrator.
- Fully modular and scalable architecture.

CUSTOMER SEGMENTS



Potential Users

- Search & Rescue Teams
- Infrastructure Inspection
- Smart Cities
- Agriculture Monitoring
- Security & Surveillance
- Research & Education

CHANNELS



- AirSim Simulation Platform
- Autonomous Drone Systems
- Smart City Applications
- Research & Academic Projects

KEY RESOURCES



Your paragraph text

- 1.AirSim Simulator
- 2.YOLOv8 Model
- 3.Dueling DQN Agent
- 4.PyTorch
- 5.OpenCV
- 6.Drone Camera
- 7.Training Dataset

COST STRUCTURE

- ✓ Model Training.
- ✓ GPU Computation.
- ✓ Dataset Collection.
- ✓ AirSim Simulation.
- ✓ Software Development.
- ✓ Model Evaluation.

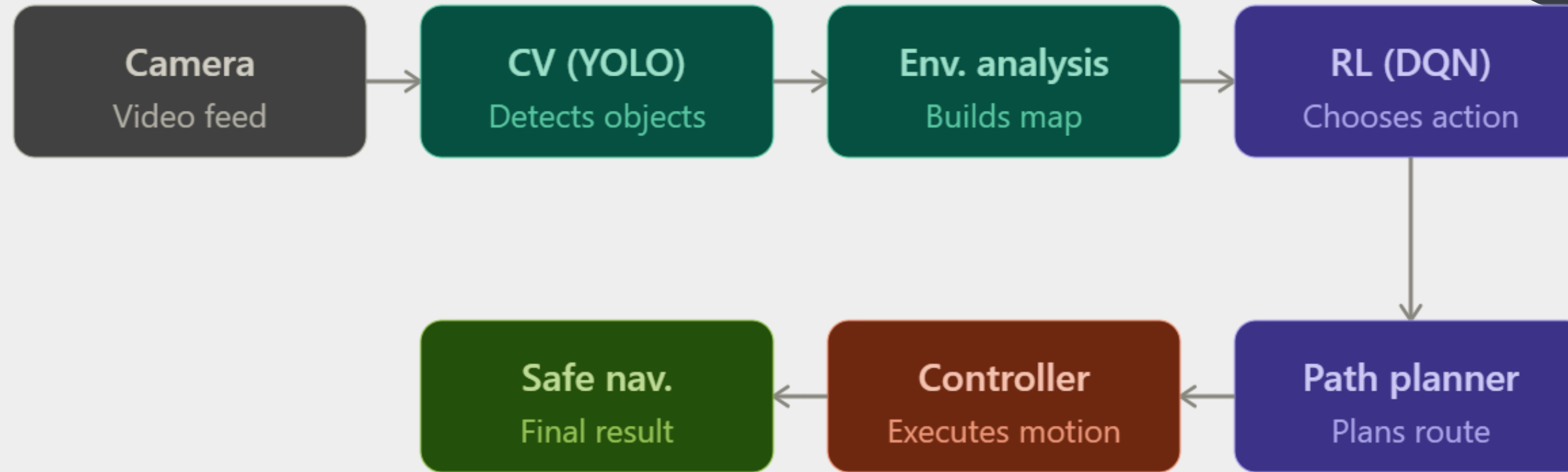


EXPECTED IMPACT

- Safer autonomous flights.
- Reduced collision rate.
- Faster navigation decisions.
- Explainable AI for drone behavior.
- Higher mission success rate.
- Scalable intelligent drone platform.

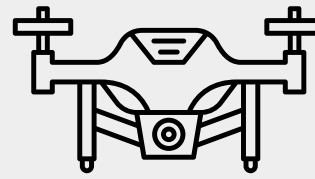


Proposed Solution



Our system combines Computer Vision, Reinforcement Learning, and Intelligent Path Planning to achieve fully autonomous drone navigation.

Camera Module



The drone is equipped with an onboard RGB camera that continuously captures images of the surrounding environment during flight. These images serve as the primary source of information for the entire autonomous navigation system.



How It Works

- The camera captures **RGB frames** continuously while the drone is flying.
- Each frame **represents a snapshot** of the drone's current surroundings.
- The captured images are immediately forwarded to the Vision Service **for object detection** and scene understanding.
- This process is repeated **many times per second**, allowing the drone to continuously monitor changes in its environment.

Why Is It Important?

The camera acts as the eyes of the drone. Without visual input, the drone cannot:

1. Detect obstacles.
2. Recognize people or vehicles.
3. Estimate its surroundings.
4. Make intelligent navigation decisions.

We face problems related to Camera

Challenge	Solution
Legal & Regulatory Constraints	Used AirSim instead of real-world flights.
Dataset & Segmentation Issues	Updated Path Planner with <i>category = classify_detection(label)</i> so segementation is not a problem anymore
Simulation Limitations	Tested under multiple simulated environments.

Vision Service (Object Detection)

The Vision Service is responsible for processing the captured camera frames and understanding the surrounding environment using Computer Vision techniques. It analyzes every frame in real time and identifies important objects that may affect the drone's navigation.

Responsibilities

- Receive RGB frames from the camera.
- Detect surrounding objects using YOLO.
- Classify detected objects (Person, Vehicle, Tree, Building...).
- Generate Bounding Boxes.
- Calculate Confidence Scores.
- Send the detection results to the Reinforcement Learning Agent.



We faced problems related to CV



Challenge	Solution
No public dataset matched our drone detection task.	Generated a custom dataset using AirSim.
YOLO misclassified objects and missed some obstacles.	Improved annotations, expanded the dataset, and retrained the model.
Running YOLO slowed the control loop.	Used parallel processing for faster inference.

The Environment Analysis module interprets the output of the Vision Service and transforms raw object detections into meaningful environmental information that can be used for intelligent navigation.

Main Tasks

- Analyze detected objects.
- Classify obstacle types.
- Estimate obstacle risk.
- Determine obstacle positions.
- Prepare environmental information for the decision-making system.



YOLO

Person

Confidence = 98%

BBox Width = 180 px

ENV analysis

Object = Person

Risk = High

Position = Front

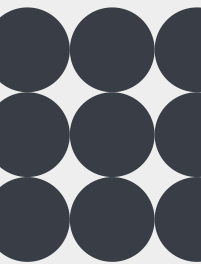
Very Close

We faced problems related to Env analysis

Challenge	Solution
Detection results were inconsistent and difficult for the RL Agent to interpret.	Converted detections into normalized state features (distance, size, position, risk).
The drone initially treated all obstacles with the same priority.	Introduced class-based risk analysis to prioritize people over vehicles and structures.

State Construction

The detected objects alone are not enough for the drone to make decisions.
The system combines multiple sources of information into a single 10-dimensional state vector, which represents the current situation of the drone.



Input to State

Drone Position
(x,y,z)

Drone Velocity
(vx,vy,vz)

Angles
(roll, pitch, yaw)

Distance to target

Output

10-Dimensional State Vector



RL Agent

Features Included

Relative Target Position
Drone Velocity
Drone Orientation (Yaw)
Obstacle Information
Risk Categories
Mission Progress

We faced problems related to State Construction

Challenge	Solution
The RL Agent received different state formats during navigation.	Built a consistent state construction function for all mission types.
The state lacked important navigation information.	Added yaw, target position, and velocity to the state.
Incomplete state information reduced RL training stability.	Used a normalized state representation to improve learning and decision-making.

Reinforcement Learning Agent



Overview

The Reinforcement Learning (RL) Agent serves as the decision-making brain of the autonomous drone. It analyzes the current state of the environment and selects the most appropriate action to safely navigate toward the target while avoiding obstacles.

Why Reinforcement Learning?

- Makes decisions autonomously.
- Adapts to dynamic environments.
- Learns optimal navigation policies.
- Reduces collision probability.

Input

1. Drone Position
2. Target Position
3. Velocity
4. Yaw Angle
5. Detected Objects
6. Mission Progress



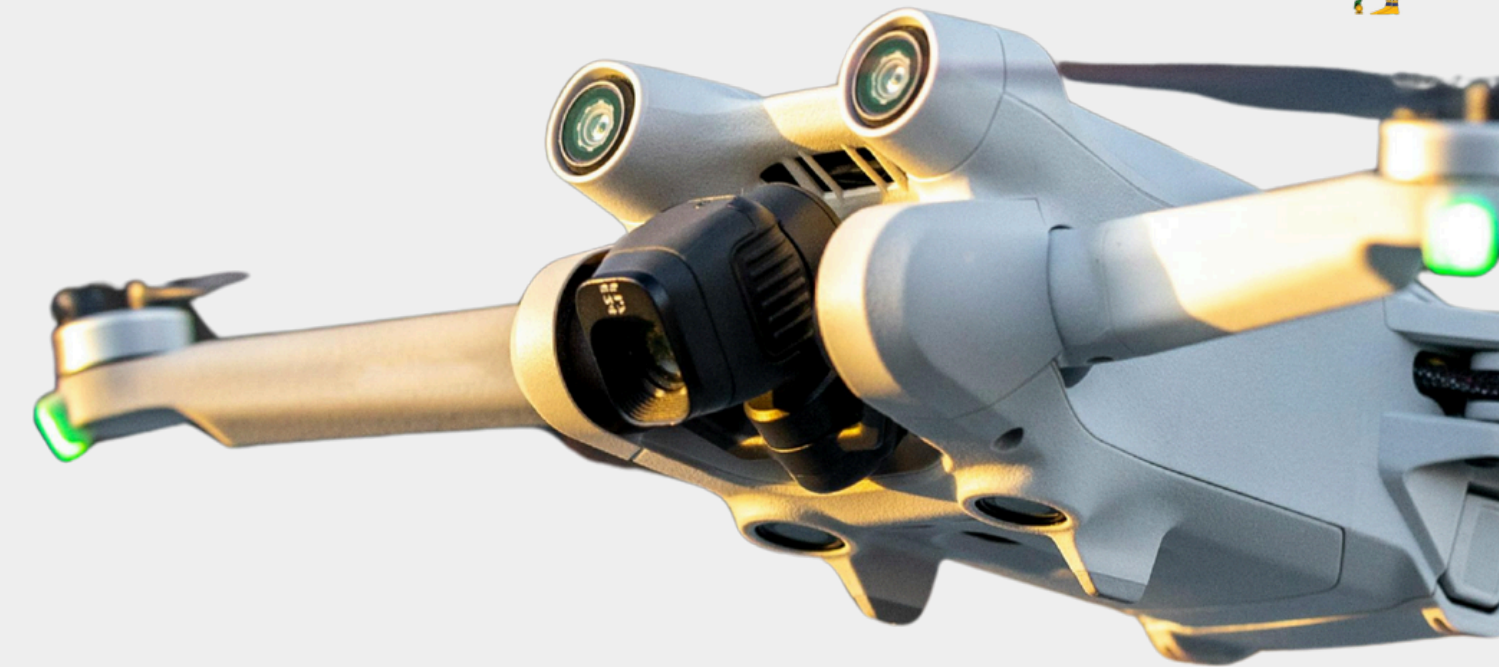
Output

1. Move Forward
2. Turn Left
3. Turn Right
4. Hover

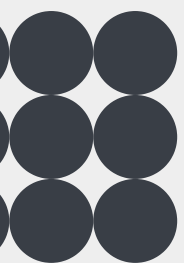
Which one ? PPO or DQN

Why PPO?

- ✓ Handles Continuous Actions
 - Generates smooth drone movement commands (vx, vy, vz).
- ✓ Stable Training
 - Uses a clipping mechanism to prevent unstable policy updates.
- ✓ Efficient Performance
 - Achieves high performance with lower computational cost than TRPO.
- ✓ Reliable Implementation
 - Implemented using the Stable-Baselines3 library.



$$R_t = R_{\text{prog}} + R_{\text{time}} + R_{\text{smooth}} + R_{\text{safety}} + R_{\text{sparse}}$$



We faced problems related to RL



Challenge	Solution
The RL agent exploited the reward function by hovering or flying in circles instead of completing the mission.	Redesigned the reward function to reward meaningful progress toward the target.
Sparse rewards slowed learning, while dense rewards encouraged local optimum behavior around obstacles.	Combined intermediate rewards with goal-based rewards to improve learning stability.
Excessive collision penalties discouraged the drone from exploring the environment.	Balanced the penalty values to promote safe but active navigation.
Balancing speed, obstacle avoidance, and smooth flight produced conflicting objectives.	Carefully tuned reward weights to achieve an optimal trade-off between all objectives.

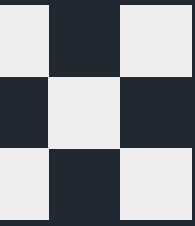
Path Planner

Role

The Path Planner acts as the drone's safety layer, ensuring that every movement is safe before execution.

How It Works

- ✓ Receives object detections from YOLO.
- ✓ Evaluates obstacle risk based on object type.
- ✓ Checks if the RL action is safe.
- ✓ Modifies unsafe decisions when necessary.
- ✓ Sends the final safe command to the Drone Controller.





Drone Controller

Role

The Drone Controller is responsible for communicating with the drone and executing all flight commands safely.

Responsibilities

- ✓ Connects to the drone (Carla).
- ✓ Receives movement commands from the Path Planner.
- ✓ Executes drone movements.
- ✓ Reads the drone position and orientation.
- ✓ Controls takeoff, landing, hover, and rotation.



Function	Description
connect()	Establishes connection with the drone.
takeoff()	Initiates drone takeoff.
land()	Safely lands the drone.
hover()	Keeps the drone stable in its current position.
move()	Moves the drone in the specified direction and speed.
rotate_yaw()	Rotates the drone left or right.
get_pose()	Retrieves the current drone position (X, Y, Z).
get_yaw()	Retrieves the drone's current heading (Yaw angle).

Drone Performance



Metric	Represents
mAP@50 = 0.696	YOLO detection accuracy
Collisions = 0	Safe autonomous navigation
Sustained Follow = 214 Steps	Stable target-following performance
Object Classes = 5	Number of detectable object categories

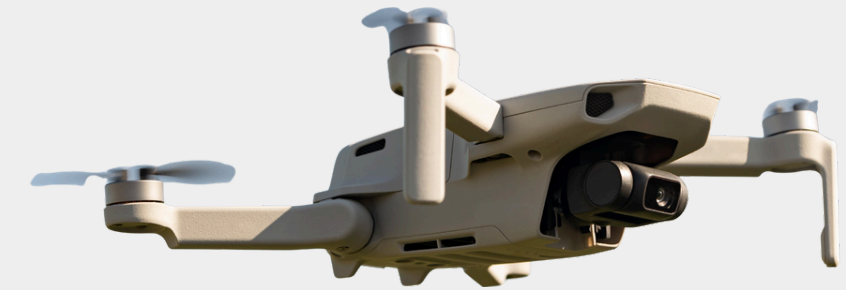
Simulation Timeeeeeeee



Improvements in Future

1. improve Model Performance

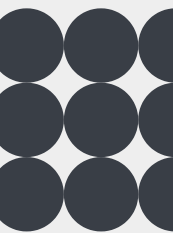
Train the model on more instances and decrease the error loss

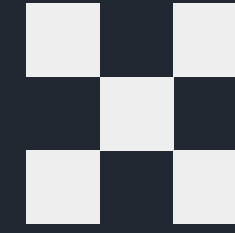


2. Deploy the System on a Real Drone

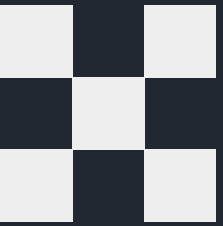
Validate the system in real-world environments instead of simulation only.

This will evaluate its performance under real weather, lighting, and sensor conditions.





Finally, we would like to thank our instructor,
Eng. Sherif Salem, for his continuous support and
guidance throughout our DEPI journey.



***THANK YOU ALL, wish us a
safe flight>>***

